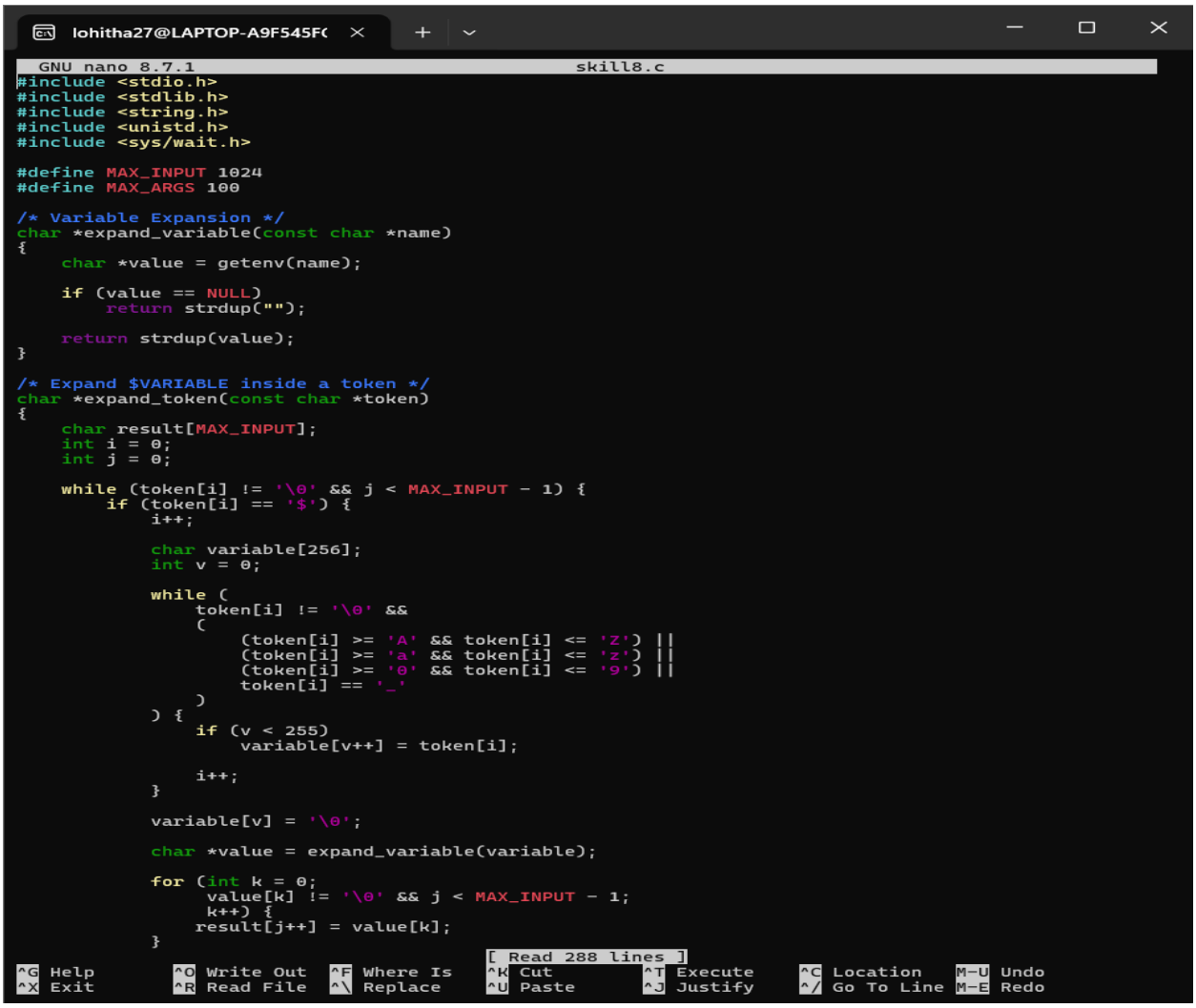
OS & SP

Name : S LOHITHA **Roll No :** 2520030109

SKILL – 08

1.Skill8.c



lohitha27@LAPTOP-A9F545F(

GNU nano 8.7.1 skill8.c

```
            free(value);
        } else {
            result[j++] = token[i++];
        }
    }

    result[j] = '\0';
    return strdup(result);
}

/* Parsing */
int parse_command(char *input, char *args[])
{
    int argc = 0;
    char *token = strtok(input, " \t");

    while (token != NULL && argc < MAX_ARGS - 1) {
        args[argc++] = token;
        token = strtok(NULL, " \t");
    }

    args[argc] = NULL;
    return argc;
}

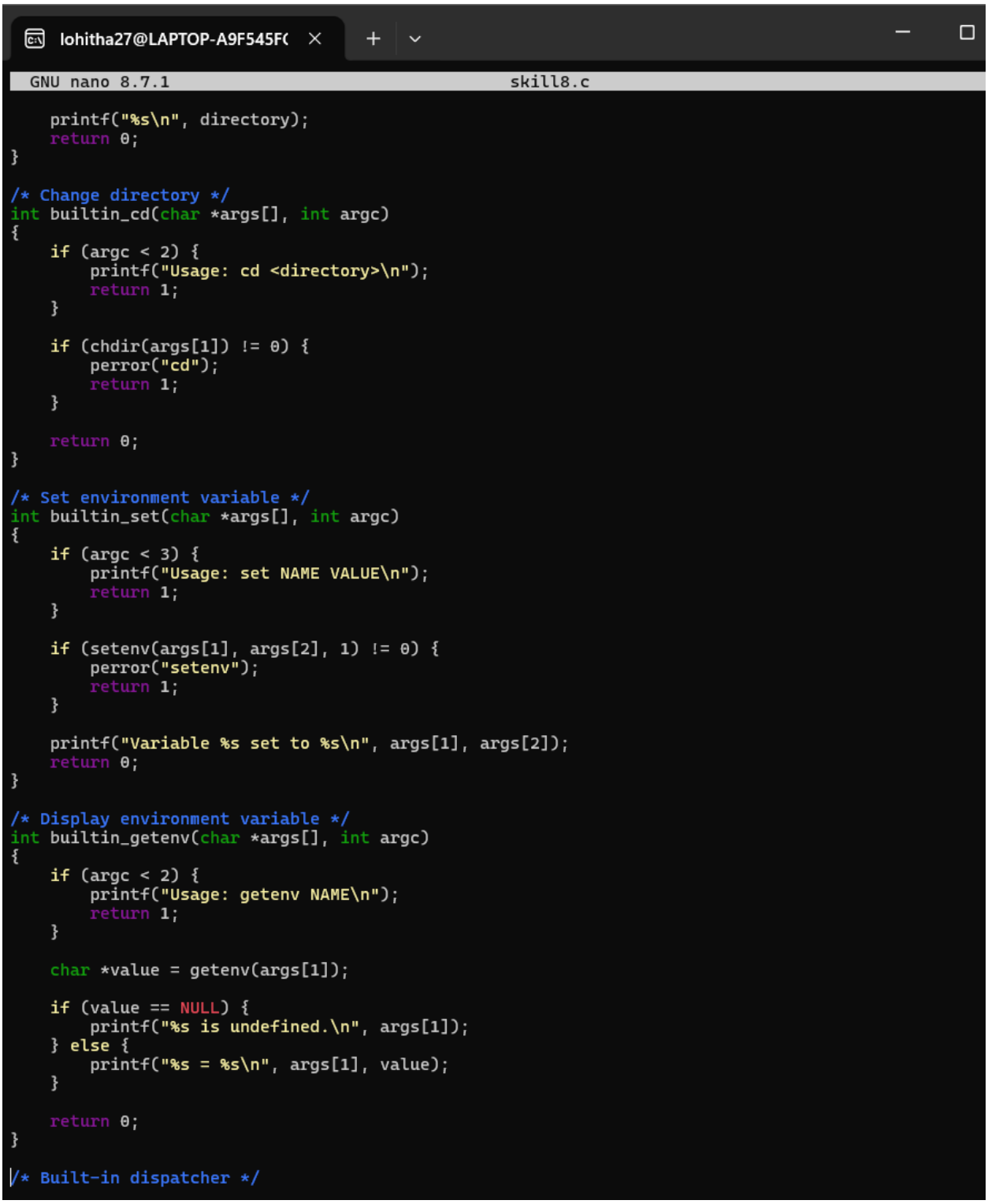
/* Built-in help */
void builtin_help(void)
{
    printf("\nBuilt-in commands:\n");
    printf(" cd <directory>  - change directory\n");
    printf(" pwd             - show current directory\n");
    printf(" set NAME VALUE  - set environment variable\n");
    printf(" getenv NAME     - display variable\n");
    printf(" help            - show help\n");
    printf(" exit            - exit program\n\n");
}

/* Display current directory */
int builtin_pwd(void)
{
    char directory[MAX_INPUT];

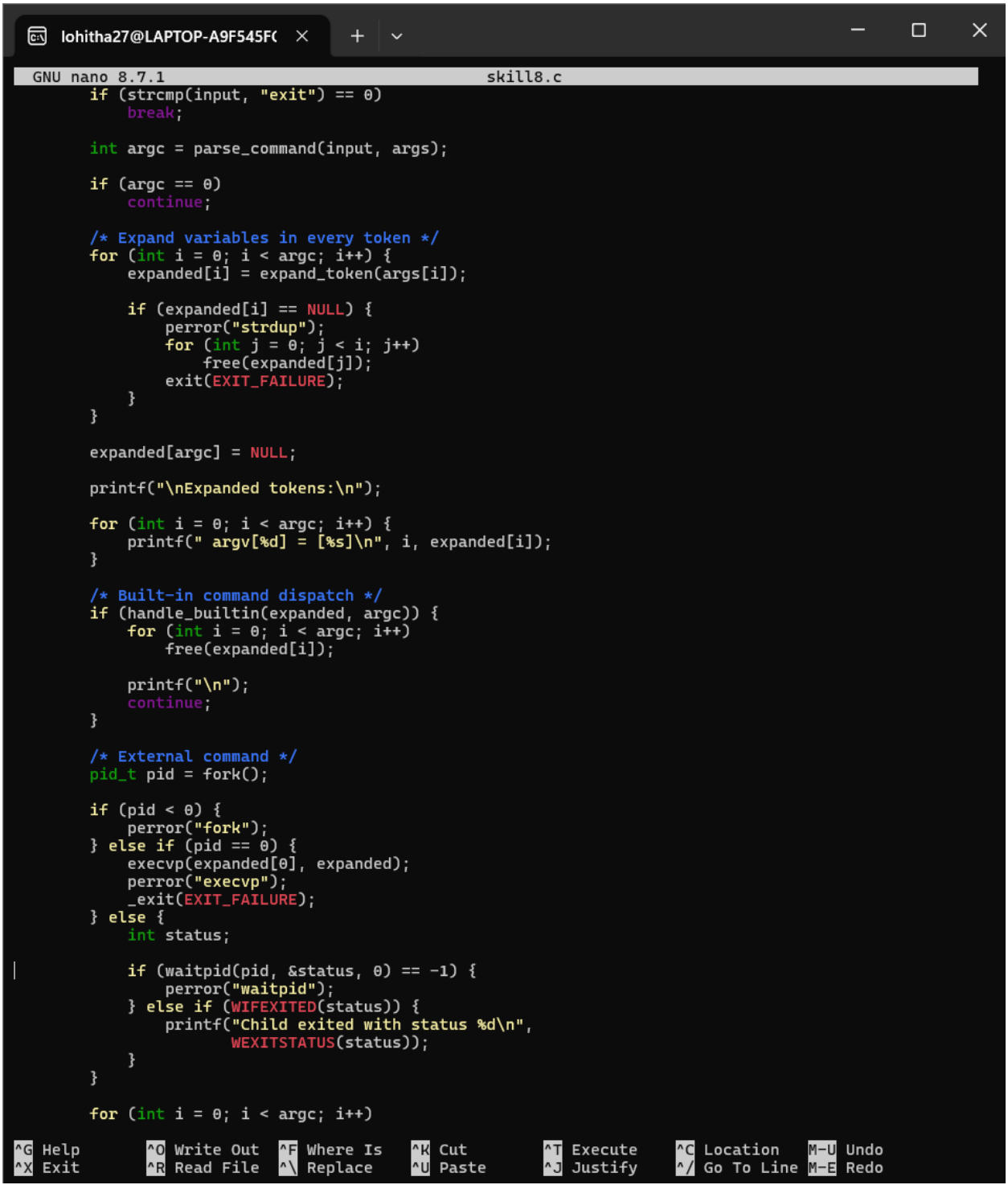
    if (getcwd(directory, sizeof(directory)) == NULL) {
        perror("getcwd");
        return 1;
    }

    printf("%s\n", directory);
    return 0;
}

/* Change directory */
int builtin_cd(char *args[], int argc)
{
    if (argc < 2) {
        printf("Usage: cd <directory>\n");
        return 1;
```


lohitha27@LAPTOP-A9F545F(
GNU nano 8.7.1 skill8.c
printf("%s\n", directory);
return 0;
}
/* Change directory */
int builtin_cd(char *args[], int argc)
{
if (argc < 2) {
printf("Usage: cd <directory>\n");
return 1;
}
if (chdir(args[1]) != 0) {
perror("cd");
return 1;
}
return 0;
}
/* Set environment variable */
int builtin_set(char *args[], int argc)
{
if (argc < 3) {
printf("Usage: set NAME VALUE\n");
return 1;
}
if (setenv(args[1], args[2], 1) != 0) {
perror("setenv");
return 1;
}
printf("Variable %s set to %s\n", args[1], args[2]);
return 0;
}
/* Display environment variable */
int builtin_getenv(char *args[], int argc)
{
if (argc < 2) {
printf("Usage: getenv NAME\n");
return 1;
}
char *value = getenv(args[1]);
if (value == NULL) {
printf("%s is undefined.\n", args[1]);
} else {
printf("%s = %s\n", args[1], value);
}
return 0;
}
/* Built-in dispatcher */


lohitha27@LAPTOP-A9F545F(
GNU nano 8.7.1 skill8.c
int handle_builtin(char *args[], int argc)
{
if (argc == 0)
return 1;
if (strcmp(args[0], "help") == 0) {
builtin_help();
return 1;
}
if (strcmp(args[0], "pwd") == 0) {
builtin_pwd();
return 1;
}
if (strcmp(args[0], "cd") == 0) {
builtin_cd(args, argc);
return 1;
}
if (strcmp(args[0], "set") == 0) {
builtin_set(args, argc);
return 1;
}
if (strcmp(args[0], "getenv") == 0) {
builtin_getenv(args, argc);
return 1;
}
return 0;
}
/* Main */
int main(void)
{
char input[MAX_INPUT];
char *args[MAX_ARGS];
char *expanded[MAX_ARGS];
printf("==========\n");
printf(" OSSP PROGRAM 8\n");
printf(" Variable Expansion + Built-ins\n");
printf("==========\n");
printf("\nType 'help' for commands.\n\n");
while (1) {
printf("shell8> ");
fflush(stdout);
if (fgets(input, sizeof(input), stdin) == NULL)
break;
input[strcspn(input, "\n")] = '\0';
if (strlen(input) == 0)
continue;


lohitha27@LAPTOP-A9F545F(
GNU nano 8.7.1 skill8.c
if (strcmp(input, "exit") == 0)
break;
int argc = parse_command(input, args);
if (argc == 0)
continue;
/* Expand variables in every token */
for (int i = 0; i < argc; i++) {
expanded[i] = expand_token(args[i]);
if (expanded[i] == NULL) {
perror("strdup");
for (int j = 0; j < i; j++)
free(expanded[j]);
exit(EXIT_FAILURE);
}
}
expanded[argc] = NULL;
printf("\nExpanded tokens:\n");
for (int i = 0; i < argc; i++) {
printf(" argv[%d] = [%s]\n", i, expanded[i]);
}
/* Built-in command dispatch */
if (handle_builtin(expanded, argc)) {
for (int i = 0; i < argc; i++)
free(expanded[i]);
printf("\n");
continue;
}
/* External command */
pid_t pid = fork();
if (pid < 0) {
perror("fork");
} else if (pid == 0) {
execvp(expanded[0], expanded);
perror("execvp");
_exit(EXIT_FAILURE);
} else {
int status;
if (waitpid(pid, &status, 0) == -1) {
perror("waitpid");
} else if (WIFEXITED(status)) {
printf("Child exited with status %d\n",
WEXITSTATUS(status));
}
}
for (int i = 0; i < argc; i++)
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo

```c
        }

        /* External command */
        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
        } else if (pid == 0) {
            execvp(expanded[0], expanded);
            perror("execvp");
            _exit(EXIT_FAILURE);
        } else {
            int status;

            if (waitpid(pid, &status, 0) == -1) {
                perror("waitpid");
            } else if (WIFEXITED(status)) {
                printf("Child exited with status %d\n",
                       WEXITSTATUS(status));
            }
        }

        for (int i = 0; i < argc; i++)
            free(expanded[i]);

        printf("\n");
    }

    printf("Program 8 terminated.\n");
    return 0;
}
```

OUTPUT :

```
lohitha27@LAPTOP-A9F545FG:~/SKILL$ nano skill8.c
lohitha27@LAPTOP-A9F545FG:~/SKILL$ nano skill8.c
lohitha27@LAPTOP-A9F545FG:~/SKILL$ gcc -Wall -Wextra -g skill8.c -o skill8
lohitha27@LAPTOP-A9F545FG:~/SKILL$ ./skill8
==========================================
        OSSP PROGRAM 8
    Variable Expansion + Built-ins
==========================================

Type 'help' for commands.

shell8> valgrind --leak-check=full --show-leak-kinds=all ./skill8

Expanded tokens:
 argv[0] = [valgrind]
 argv[1] = [--leak-check=full]
 argv[2] = [--show-leak-kinds=all]
 argv[3] = [./skill8]
execvp: No such file or directory
Child exited with status 1

shell8> echo $NAME

Expanded tokens:
 argv[0] = [echo]
 argv[1] = [LAPTOP-A9F545FG]
LAPTOP-A9F545FG
Child exited with status 0

shell8>
```